



Département de Mathématiques

Filière Licence Sciences et Techniques : Sciences des Données et Informatique décisionnelle

Développement d'une application JEE pour la gestion des réservations "AL BORAQ"

Présenté par :

- Kenza BIZARAN
- Othmane BOUGHALI
- Mohamed Amine H'SAIN
- Mustapha KASSIMI
- Imane ALAOUI MODAR
- Yasser BICHRI

Encadré par :

- Mme Sbai Hanae

Livré le : 09/01/2025

Avant-Propos

Ce rapport documente le développement d'une application web de gestion des réservations de trains, intitulée "AL BORAQ". Ce projet s'inscrit dans le cadre de notre formation en Licence Sciences et Techniques, spécialité Sciences des Données et Informatique Décisionnelle.

Il reflète notre ambition de mettre en pratique les concepts étudiés en classe, tout en relevant les défis techniques inhérents au développement d'un système complexe. L'application a été conçue en utilisant Jakarta EE, une technologie moderne pour les applications web, et intègre des aspects tels que la gestion des utilisateurs, les réservations, et la personnalisation de l'interface utilisateur.

Nous avons tenu à mettre l'accent sur une architecture évolutive, tout en garantissant une interface intuitive pour l'utilisateur final. Ce projet représente une opportunité précieuse d'explorer les différentes facettes du développement logiciel, en alliant théorie et pratique. Ce projet s'inscrit dans le cadre de notre formation en Licence Sciences et Techniques, spécialité Sciences des Données et Informatique Décisionnelle.

Table des matières

1	Introduction	4
1.1	Contexte et Motivation	4
1.2	Problématique	4
1.3	Portée	4
2	Méthodologie	5
2.1	Outils et Technologie	5
2.2	Architecture	5
2.3	Diagrammes :	6
2.4	Diagramme de cas d'utilisation :	6
2.5	Diagramme des classes :	7
2.6	Développement	7
3	Résultats	23
3.1	Tests :	23
3.1.1	Interface 1 :	23
3.1.2	Interface 2 :	24
3.1.3	Interface 3 :	25
3.1.4	Interface 4 :	25
3.1.5	Interface 5 :	26
3.1.6	Interface 6 :	26
3.1.7	Interface 7 :	27
3.1.8	Interface 8 :	28
3.1.9	Interface 9 :	28
4	Discussion	29
4.1	Points Forts de notre Projet :	29
4.2	Perspectives	29
4.3	Défis rencontrés	30
5	Conclusion	30

1 Introduction

1.1 Contexte et Motivation

La réservation de trains est une activité importante pour de nombreuses personnes, notamment celles qui effectuent régulièrement des trajets domicile-travail ou domicile-études. Grâce aux avancées technologiques, il est devenu plus facile de simplifier ce processus via des solutions numériques adaptées.

Dans ce cadre, notre projet vise à concevoir une application web de réservation de trains inspirée des systèmes existants, en utilisant les technologies Jakarta Enterprise Edition (*Jakarta EE*). Ce projet constitue également une opportunité pour nous d'apprendre et d'appliquer des concepts fondamentaux du développement logiciel.

1.2 Problématique

En tant qu'étudiants effectuant chaque jour des trajets pour nos besoins académiques et personnels, nous avons été confrontés à des situations telles que :

- ✓ afficher les trains dont leur départ est uniquement après l'heure actuelle.
- ✓ afficher que les places restantes sont limités.
- ✓ Gestion des profils utilisateurs.
- ✓ Paiement en ligne.

Ces expériences nous ont incités à explorer le fonctionnement des systèmes de réservation actuels et à mieux comprendre leurs mécanismes.

1.3 Portée

Gestion des utilisateurs : Création de comptes, authentification, et gestion des profils.

Consultation des horaires : Affichage des horaires des trains en fonction des itinéraires sélectionnés.

Réservation de billets : Sélection des trajets, choix des places et confirmation des réservations.

Suivi des réservations : Consultation, modification ou annulation des réservations existantes.

Interface utilisateur : Conception d'une interface simple et intuitive pour faciliter l'interaction.

Gestion back-end : Développement des fonctionnalités côté serveur pour traiter les requêtes et gérer les données. Ces points assurent que le projet répond aux besoins essentiels d'une application de réservation de trains.

2 Méthodologie

2.1 Outils et Technologie

Langage de programmation :

- Java : *Principal langage pour développer les applications Jakarta EE.*

Serveur d'application :

- Apache Tomcat : *Pour déployer et exécuter l'application.*

Build System :

- Maven : *Pour gérer les dépendances et le cycle de vie du projet.*

Frameworks et API :

- Jakarta Servlet : *Pour gérer les requêtes HTTP.*

Base de données :

- MySQL : *Pour stocker les données.*

2.2 Architecture

L'architecture de notre application suit une structure MVC (Modèle-Vue-Contrôleur) déployée sur le serveur d'application Apache Tomcat.

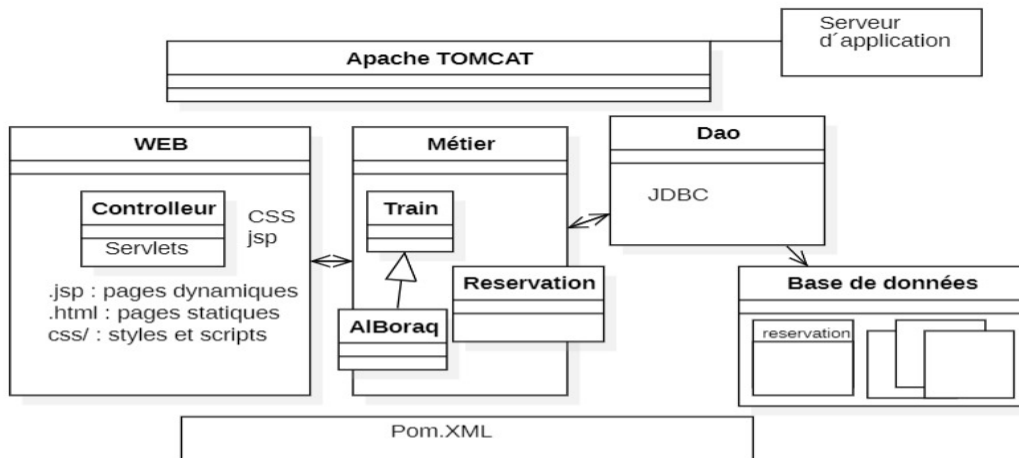
La couche WEB gère l'interface utilisateur avec des pages statiques (.html) et dynamiques (.jsp), enrichies par des styles CSS. Les requêtes sont traitées par la Servlet, qui agit comme contrôleur.

La couche Métier encapsule la logique applicative, comprenant des classes comme Train, Reservation, et AlBoraq, qui interagissent avec les données.

La couche DAO (Data Access Object) assure la communication avec la base de données via JDBC, en utilisant le modèle de données Reservation.

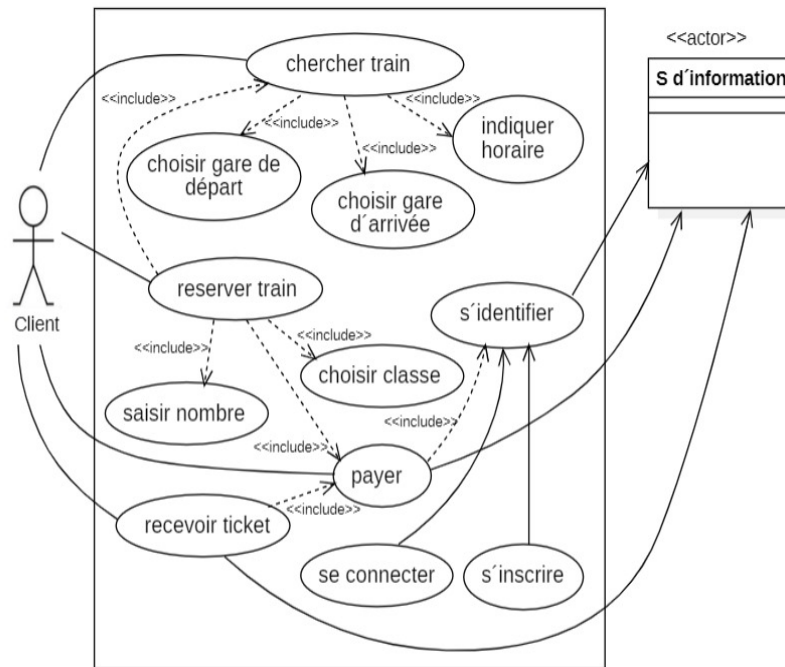
Les dépendances et configurations sont gérées à l'aide du fichier pom.xml, garantissant la portabilité et la modularité de l'application. Cette organisation assure une séparation claire des responsabilités et facilite la maintenance.

Pour mieux clarifier les aspects du projet et en offrir une vue d'ensemble, voici une architecture globale qui présente les technologies et outils utilisés.

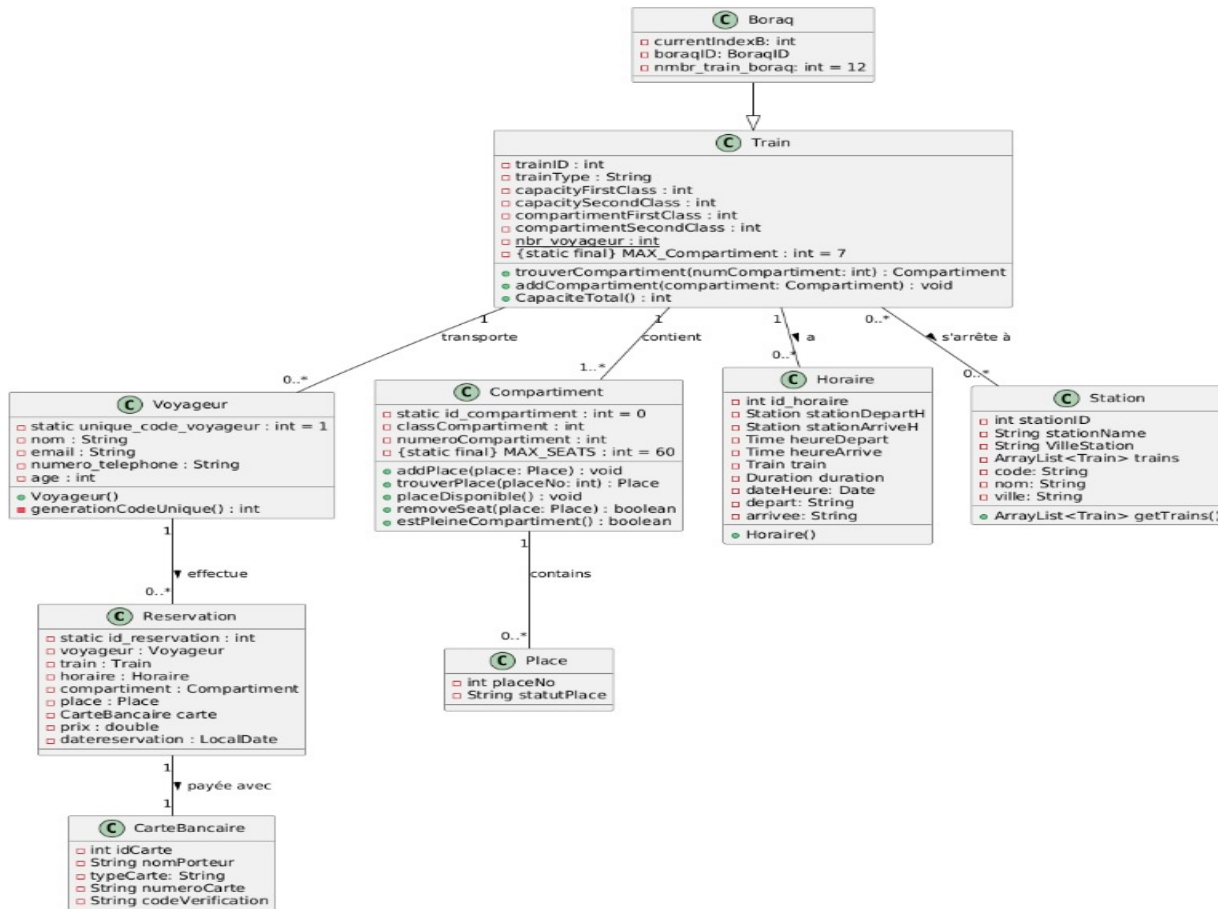


2.3 Diagrammes :

2.4 Diagramme de cas d'utilisation :



2.5 Diagramme des classes :



2.6 Développement

Le développement du site est conçu en deux parties principales : **le front-end** et **le back-end**.

→ Le back-end qui fonctionne en arrière-plan pour traiter les données et assurer le fonctionnement de l'application. repose sur une architecture en trois couches essentielles :

- **la couche METIER**

Cette couche est responsable de l'application des règles fonctionnelles spécifiques à l'activité du système. Elle encapsule toute la logique métier, garantissant ainsi une séparation claire entre les données brutes (gérées par la couche DAO) et leur présentation (assurée par la couche web). Son rôle est d'assurer la cohérence des opérations effectuées.

Voici une présentation des classes principales ainsi que leurs fonctionnalités respectives :

* Classe Train :

Cette classe représente un train avec ses compartiments et ses voyageurs, tout en fournissant des méthodes pour gérer la capacité et l'occupation du train

```
1 package SDIDproject;
2
3 import lombok.Data;
4
5 import java.util.ArrayList;
6 import java.util.Iterator;
7 @Data
8 public class Train {
9     private int trainID;
10    private String trainType;
11    private int capacityFirstClass;
12    private int capacitySecondClass;
13    private int compartmentFirstClass ;
14    private int compartmentSecondClass ;
15    private static int nbr_voyageur;
16    private ArrayList<Voyageur> voyageurs;
17    private ArrayList<Compartment> compartments;
18    private static final int MAX_Compartment = 7;
19    public Train(String trainType,int trainID) {
20        this.trainType = trainType;
21        this.trainID = trainID;
22    }
23    public Train(String trainType, int capacityFirstClass, int capacitySecondClass, int compartmentFirstClass, int compartmentSecondClass) {...}
24    public Compartment trouverCompartment(int numCompartment) {...}
25    public void addCompartment(Compartment compartment) throws TrainException {...}
26    public int CapaciteTotal(){...}
27    public void addVoyageur(Voyageur voyageur) throws TrainException {...}
28    public void deleteVoyageur(Voyageur voyageur) throws TrainException {...}
29    if (voyageurs.remove(voyageur)) {...}
30    public void afficherVoyageur(){...}
31    @Override
32    public String toString() {...}
33 }
```

* Classe Boraq :

Cette classe représente un train "Boraq" qui a des propriétés spécifiques, notamment un identifiant unique généré à partir d'une énumération, tout en héritant des caractéristiques communes d'un train via la classe Train.

```
1 package SDIDproject;
2
3 import lombok.Data;
4
5 @Data
6 public class Boraq extends Train {
7     private static int currentIndexB = 0;
8     private BoraqID boraqID;
```



```

9     private final static int nmbr_train_boraq = 12;
10
11     public Boraq() {
12         super("Boraq", 80, 400, 2, 5);
13         this.boraqID = BoraqID.values()[currentIndexB];
14         this.setTrainID(this.boraqID.getValue());
15         currentIndexB = (currentIndexB + 1) % BoraqID.values().length;
16     }

```

* Classe Compartiment :

La classe Compartiment représente un compartiment dans un train, avec plusieurs fonctionnalités liées à la gestion des places dans ce compartiment.

```

1  package SDIDproject;
2  import java.util.ArrayList;
3  public class Compartiment {
4      public static final int MAX_SEATS = 60;
5      private static int id_compartiment=0;
6      private int classCompartiment;
7      private int numeroCompartiment;
8      private ArrayList<Place> places;
9      public Compartiment(int classCompartiment, int numeroCompartiment) throws
10         CompartimentException{
11         if (classCompartiment != 1 && classCompartiment != 2) {
12             throw new CompartimentException("La classe du compartiment doit
13             tre 1 (Premi re Classe) ou 2 (Deuxi me Classe).");
14         }
15         id_compartiment++;
16         this.classCompartiment = classCompartiment;
17         this.numeroCompartiment = numeroCompartiment;
18         this.places = new ArrayList<>();
19     }
20     public void addPlace(Place place) {
21         if (place.estDisponible()) {
22             places.add(place);
23         } else {
24             System.out.println("Le compartiment est plein . Voici les places li
25             bres : ");
26             placeDisponible();
27         }
28     }
29     public Place trouverPlace(int placeNo) {
30         for (Place place : places) {
31             if (place.getPlaceNo() == placeNo) {
32                 return place;
33             }
34         }
35         return null;
36     }
37     public void placeDisponible() {
38         System.out.println("Si ges disponibles dans le compartiment " +
39         numeroCompartiment + ":");
40         for (Place place : places) {
41             if (place.getStatutPlace().equals("Libre")) {
42                 System.out.println(place.toString());
43             }
44         }
45     }

```

```

41     }
42     public boolean removeSeat(Place place) {
43         return places.remove(place);
44     }
45     public boolean estPleineCompartment() {
46         return places.size() == MAX_SEATS;
47     }
48     public ArrayList<Place> getPlaces() {
49         return places;
50     }
51     public void setPlaces(ArrayList<Place> places) throws CompartimentException
52     {
53         if (places.size() > MAX_SEATS) {
54             throw new CompartimentException("Un compartiment ne peut pas
55             contenir plus de " + MAX_SEATS + " places ");
56         }
57         this.places = places;
58     }
59     public int getClassCompartment() {
60         return classCompartment;
61     }
62     public void setClassCompartment(int classCompartment) throws CompartimentException{
63         if (classCompartment != 1 && classCompartment != 2) {
64             throw new CompartimentException("La classe du compartiment doit
65             tre 1 (Premi re Classe) ou 2 (Deuxi me Classe).");
66         }
67         this.classCompartment = classCompartment;
68     }
69     public int getNumeroCompartment() {
70         return numeroCompartment;
71     }
72     public void setNumeroCompartment(int numeroCompartment) {
73         this.numeroCompartment = numeroCompartment;
74     }
75 }

```

* Classe Place :

Cette classe est utilisée pour gérer individuellement les places dans un compartiment de train, avec un statut indiquant si la place est occupée ou libre.

```

1 package SDIDproject;
2 public class Place {
3     private int placeNo;
4     private String statutPlace;
5     public String getStatutPlace() {
6         return statutPlace;
7     }
8     public Place(int placeNo) {
9         this.placeNo = placeNo;
10        this.statutPlace = "Libre";
11    }
12    public void setStatutPlace(String statutPlace) {
13        this.statutPlace = statutPlace;
14    }
15    public int getPlaceNo() {
16        return placeNo;
17    }
18 }

```

```

17     }
18     public void setPlaceNo(int placeNo) {
19         this.placeNo = placeNo;
20     }
21     public boolean estDisponible() {
22         return statutPlace.equals("Libre");
23     }
24     @Override
25     public String toString() {
26         return "Place :" +
27             "placeNo=" + placeNo +
28             ", statutPlace='" + statutPlace;
29     }
30 }

```

* Classe Billet :

La classe Billet encapsule toutes les informations pertinentes d'une réservation et fournit une méthode pour imprimer ou afficher le billet sous un format lisible.

```

1 package SDIDproject;
2
3 import lombok.Data;
4
5 import java.io.Serializable;
6 @Data
7 public class Billet implements Serializable {
8     private static int billetID=0;
9     private Reservation reservation;
10    public Billet(Reservation reservation) {
11        billetID++;
12        this.reservation = reservation;
13    }
14
15
16    public String printBilletsPDF() {
17        return "==== Ticket PDF ===== \n" + "Ticket #" + billetID +
18            "\n Voyageur: " + reservation.getVoyageur().getNom() +
19            "\n Train: " + reservation.getTrain().getTrainType() + reservation.getTrain().getTrainID() +
20            "\n Horaire: " + reservation.getHoraire().toString() +
21            "\n Compartiment: " + reservation.getCompartiment().getNumeroCompartiment() +
22            "\n Place: " + reservation.getPlace().getPlaceNo() +
23            "\n Prix: " + reservation.getPrix() + " DH\n" +
24            "===== ";
25    }
26 }

```

* Classe Reservation :

La classe Reservation est responsable de la gestion des réservations de trains, y compris la réservation et l'annulation de places, ainsi que le suivi des informations de paiement et des détails du voyage.

```

1 package SDIDproject;

```

```

2 import java.time.LocalDate;
3 @Data
4 public class Reservation {
5     private static int id_reservation;
6     private Voyageur voyageur;
7     private Train train;
8     private Horaire horaire;
9     private Compartiment compartiment;
10    private Place place;
11    private CarteBancaire carte;
12    private double prix;
13    private LocalDate datereservation;
14
15    public Reservation() {
16        super();
17    }
18    public Reservation(Voyageur voyageur, Train train, Horaire horaire, Compartiment compartiment, Place place, double prix, CarteBancaire carte, LocalDate datereservation) {
19        id_reservation++;
20        this.voyageur = voyageur;
21        this.train = train;
22        this.horaire = horaire;
23        this.compartiment = compartiment;
24        this.place = place;
25        this.prix = prix;
26        this.carte = carte;
27        this.datereservation = datereservation;
28    }
29
30    public void reserverPlace() throws ReservationException {...}
31    public void annulerReservation() throws Exception { }
32    @Override
33    public String toString() { }
34 }

```

* Classe Trajet :

La classe Trajet permet de gérer le parcours d'un train entre plusieurs stations. Elle offre des méthodes pour ajouter des stations, afficher toutes les stations du trajet et gérer la liste des stations de manière dynamique grâce à l'utilisation de LinkedList.

```

1 package SDIDproject;
2
3 import lombok.Data;
4
5 import java.util.Iterator;
6 import java.util.LinkedList;
7 @Data
8 public class Trajet {
9     private int id_trajet;
10    private Station stationDepartT;
11    private Station stationArriveT;
12    private Train train;
13    private LinkedList<Station> stations;
14    public Trajet(int id_trajet, Station stationDepartT, Station stationArriveT, Train train) {

```

```

15     this.id_trajet = id_trajet;
16     this.train = train;
17     this.stations=new LinkedList<>();
18     stations.addFirst(stationDepartT);
19     stations.addLast(stationArriveT);
20 }
21 public void ajouterStation(Station station) {...}
22     public void afficherStations(){    }
23 }

```

* Classe Station :

La classe Station permet de gérer les trains qui s'arrêtent à une station, d'ajouter ou de supprimer des trains, et d'afficher la liste des trains présents dans la station. Elle sert de base pour organiser et manipuler les informations relatives aux stations et aux trains.

```

1 package SDIDproject;
2 import lombok.Data;
3 import java.util.ArrayList;
4 import java.util.Iterator;
5 @Data
6 public class Station {
7     private int stationID;
8     private String stationName;
9     private String VilleStation;
10    private ArrayList<Train> trains;
11    public Station(int stationID, String stationName, String VilleStation) {
12        this.stationID = stationID;
13        this.stationName = stationName;
14        this.VilleStation = VilleStation;
15        this.trains = new ArrayList<>();
16    }
17    public void addTrain(Train train) {    }
18        public void deleteTrain(Train train) {    }
19        public void afficherTrains() {    }
20        @Override
21    public String toString() {
22        return "Station : " +
23            "stationID = " + stationID +
24            ", stationName = " + stationName + '\n' +
25            ", VilleStation = " + VilleStation;
26    }
27 }

```

* Classe Voyageur :

La classe Voyageur gère les informations personnelles d'un voyageur, avec un identifiant unique qui est automatiquement généré lors de la création d'un nouvel objet.

```

1 package SDIDproject;
2
3 import lombok.Data;
4
5 @Data
6 public class Voyageur {
7     private static int unique_code_voyageur =1;

```

```

8     private String nom;
9     private String email;
10    private String numero_telephone;
11    private int age;
12    private String password;
13    public Voyageur(String nom, String email, String numero, int age) {
14        unique_code_voyageur = generationCodeUnique();
15        this.nom = nom;
16        this.email = email;
17        this.numero_telephone = numero;
18        this.age = age;
19    }
20    public Voyageur(String nom, String email, String numero, int age, String password) {
21        unique_code_voyageur = generationCodeUnique();
22        this.nom = nom;
23        this.email = email;
24        this.numero_telephone = numero;
25        this.age = age;
26        this.password = password;
27    }
28    private synchronized int generationCodeUnique() {
29        return unique_code_voyageur++;
30    }
31
32    @Override
33    public String toString() {
34        return "Voyageur{" +
35            "unique_code_voyageur=" + unique_code_voyageur +
36            ", nom='" + nom + '\'' +
37            ", email='" + email + '\'' +
38            ", numero_telephone=" + numero_telephone +
39            '}';
40    }
41 }

```

* Classe Horaire :

La classe Horaire représente l'horaire d'un trajet de train. Elle contient des informations sur les horaires de départ et d'arrivée d'un train ainsi que les stations correspondantes.

```

1 package SDIDproject;
2 import java.sql.Time;
3 import java.time.Duration;
4 import java.time.LocalDateTime;
5
6 public class Horaire {
7     private int id_horaire;
8     private Station stationDepartH;
9     private Station stationArriveH;
10    private Time heureDepart;
11    private Time heureArrive;
12    private Train train;
13    private transient boolean imminent;
14    private transient double totalPrice;
15
16    public Horaire(int id, Station stationDepart, Station stationArrive, LocalDateTime heureDepart, LocalDateTime heureArrive, Train train) {

```

```

17         this.id_horaire = id;
18         this.stationDepartH = stationDepart;
19         this.stationArriveH = stationArrive;
20         this.heureDepart = Time.valueOf(heureDepart);
21         this.heureArrive = Time.valueOf(heureArrive);
22         this.train = train;
23     }
24
25     public Horaire() {}
26
27     public int getId() {    }
28     public Station getStationArrive() {    }
29     public void setStationArrive(Station stationArrive) {    }
30     public Station getStationDepart() {    }
31
32     public void setStationDepart(Station stationDepart) {    }
33     public Time getHeureDepart() {    }
34
35     public void setHeureDepart(Time heureDepart) {    }
36     public Time getHeureArrive() {    }
37     public void setHeureArrive(Time heureArrive) {    }
38     public Train getTrain() {    }
39     public void setTrain(Train train) {    }
40     public boolean isImminent() {    }
41     public void setImminent(boolean imminent) {    }
42     public double getTotalPrice() {
43         return totalPrice;
44     }
45     public void setTotalPrice(double totalPrice) {    }
46     public String getDuration() {
47         LocalDateTime depart = heureDepart.toLocalTime();
48         LocalDateTime arrive = heureArrive.toLocalTime();
49         Duration duration;
50         if (arrive.isBefore(depart)) {
51             duration = Duration.between(depart, LocalDateTime.MIDNIGHT)
52                 .plus(Duration.between(LocalTime.MIN, arrive));
53         } else {
54             duration = Duration.between(depart, arrive);
55         }
56         long hours = duration.toHours();
57         long minutes = duration.toMinutes() % 60;
58         return String.format("%02d:%02d", hours, minutes);
59     }
60     @Override
61     public String toString() {
62         return "Horaire : " +
63             "id_horaire = " + id_horaire +
64             ",\nstationDepart = " + stationDepartH.getStationName() + '\n' +
65             ",\nstationArrive = " + stationArriveH.getStationName() + '\n' +
66             ",\nheureDepart = " + heureDepart + '\n' +
67             ",\nheureArrive = " + heureArrive + '\n' +
68             ",\ntrain = " + train.getTrainType() + train.getTrainID();
69     }}

```

* Classe CarteBancaire :

La classe CarteBancaire représente un mode de paiement pour une réservation de train. Elle contient des informations concernant ce type de paiement .

```
1  /*
2   * Package SDIDproject
3   */
4  package SDIDproject;
5
6  public class CarteBancaire {
7      private int idCarte;
8      private String nomPorteur;
9      private String numeroCarte;
10     private int moisExpiration;
11     private int anneeExpiration;
12     private String codeVerification;
13
14     // Constructor
15     public CarteBancaire(int idCarte, String nomPorteur, String numeroCarte,
16         int moisExpiration, int anneeExpiration, String codeVerification) {
17         this.idCarte = idCarte;
18         this.nomPorteur = nomPorteur;
19         this.numeroCarte = numeroCarte;
20         this.moisExpiration = moisExpiration;
21         this.anneeExpiration = anneeExpiration;
22         this.codeVerification = codeVerification;
23     }
24
25     // Getters and Setters
26     public int getIdCarte() {
27         return idCarte;
28     }
29
30     public void setIdCarte(int idCarte) {
31         this.idCarte = idCarte;
32     }
33
34     public String getNomPorteur() {
35         return nomPorteur;
36     }
37
38     public void setNomPorteur(String nomPorteur) {
39         this.nomPorteur = nomPorteur;
40     }
41
42     public String getNumeroCarte() {
43         return numeroCarte;
44     }
45
46     public void setNumeroCarte(String numeroCarte) {
47         this.numeroCarte = numeroCarte;
48     }
49 }
```

- **La couche DAO** (Data Access Object) pour la gestion des interactions avec la base de données.

* Classe dbconnexion :

Elle est responsable de la gestion de la connexion à une base de données MySQL à l'aide de JDBC (Java Database Connectivity).

```

1 package Dao;
2 import java.sql.DriverManager;
3 import java.sql.SQLException;
4 import java.sql.Connection;
5
6 public class dbconnexion {
7     private static Connection connexion;
8     static {
9         try {
10             Class.forName("com.mysql.jdbc.Driver");
11         } catch (ClassNotFoundException e) {
12             // TODO Auto-generated catch block
13             e.printStackTrace();
14         }
15         try {
16             connexion=DriverManager.getConnection("jdbc:mysql://localhost:3306/
17                 Gestion_Reservation","root","");
18         } catch (SQLException e) {
19             // TODO Auto-generated catch block
20             e.printStackTrace();
21         }
22     }
23     public static Connection getConnexion() {
24         return connexion;
25     }
26     public static void setConnexion(Connection connexion) {
27         dbconnexion.connexion = connexion;
28     }
29 }

```

* Classe ImplReservation :

Cette classe agit comme un service pour gérer les différentes opérations liées aux réservations, aux voyageurs, aux trains, et aux stations, tout en interagissant directement avec la base de données pour effectuer des opérations CRUD (Create, Read, Update, Delete).

implReservation.png

```

1 package Dao;
2 import SDIDproject.*;
3 import java.sql.*;
4 import java.util.ArrayList;
5 import java.util.LinkedList;
6 public class ImplReservation implements IReservation {
7     private ArrayList<Reservation> reservations = new ArrayList<Reservation>();
8     private ArrayList<Voyageur> voyageurs = new ArrayList<Voyageur>();
9     private ArrayList<Train> trains = new ArrayList<Train>();
10    private ArrayList<Station> stations = new ArrayList<Station>();
11    @Override
12    public void ajouterVoyageur(Voyageur vg) {
13        Connection conn = dbconnexion.getConnexion(); // obtenir la connexion
14        try {
15            PreparedStatement ps = conn.prepareStatement("INSERT INTO voyageur
16                (nom, email, numero_telephone, age) VALUES (?, ?, ?, ?)");
17            ps.setString(1, vg.getNom());

```

```

17         ps.setString(2, vg.getEmail());
18         ps.setString(3, vg.getNumero());
19         ps.setInt(4, vg.getAge());
20         ps.executeUpdate();
21         System.out.println("Voyageur ajout avec succ s");
22     } catch (SQLException e) {
23         e.printStackTrace();
24     }
25 }
26 public CarteBancaire getCarteBancaireByNomPorteur(String nomPorteur) {
27     Connection conn = dbconnexion.getConnexion();
28     CarteBancaire carte = null;
29     String query = "SELECT * FROM CarteBancaire WHERE nom_porteur = ?";
30     try {
31         PreparedStatement ps = conn.prepareStatement(query);
32         ps.setString(1, nomPorteur);
33         ResultSet rs = ps.executeQuery();
34         if (rs.next()) {
35             carte = new CarteBancaire(
36                 rs.getInt("id_carte"),
37                 rs.getString("nom_porteur"),
38                 rs.getString("numero_carte"),
39                 rs.getInt("mois_expiration"),
40                 rs.getInt("annee_expiration"),
41                 rs.getString("code_verification")
42             );
43         }
44         rs.close();
45     } catch (SQLException e) {
46         e.printStackTrace();
47     }
48     return carte;
49 }
50
51 public Voyageur rechercherVoyageurByEmail(String email) {
52     Connection conn = dbconnexion.getConnexion();
53     Voyageur voyageur = null;
54     try {
55         PreparedStatement ps = conn.prepareStatement("SELECT * FROM voya
56             geur WHERE email = ?");
57         ps.setString(1, email);
58         ResultSet rs = ps.executeQuery();
59         if (rs.next()) {
60             voyageur = new Voyageur(rs.getString("nom"), rs.getString("ema
61                 il"), rs.getString("numero_telephone"), rs.getInt("age"));
62             voyageur.setCode(rs.getInt("unique_code_voyageur")); // Ensure
63                 the unique code is set
64         }
65         rs.close();
66     } catch (SQLException e) {
67         e.printStackTrace();
68     }
69     return voyageur;
70 }

```

* Classe IReservation :

L'interface IReservation définit un ensemble de méthodes que toute classe de gestion des

réservations doit implémenter. Ces méthodes couvrent des opérations liées aux voyageurs, aux horaires, aux réservations, aux trains et aux stations.

```

1 package Dao;
2 import SDIDproject.*;
3 import java.sql.SQLException;
4 import java.util.ArrayList;
5 import java.util.LinkedList;
6 import java.util.List;
7 public interface IReservation {
8     void ajouterVoyageur(Voyageur vg);
9     Voyageur rechercherVoyageurByEmail(String email);
10    CarteBancaire getCarteBancaireByNomPorteur(String nomPorteur);
11    LinkedList<Horaire> afficherhoraire(Station station_depart , Station station_arrivee);
12    void supprimerVoyageur(int id_voyageur) throws SQLException;
13    Voyageur rechercherVoyageur(String nomComple) throws SQLException;
14    ArrayList<Voyageur> afficherVoyageurs() throws SQLException;
15    void ajouterReservation(Reservation rs) throws SQLException;
16    void supprimerReservation(int id_reservation) throws SQLException;
17    ArrayList<Train> afficherTousTrains() throws SQLException;
18    ArrayList<Station> afficherToutesStations() throws SQLException;
19    double AdultPrix(Station station_depart , Station station_arrivee) throws
        SQLException;
20    double EnfantPrix (Station station_depart , Station station_arrivee) throws
        SQLException;
21    void ajouterAuthentification(String email, String password);
22    boolean verifierAuthentification(String email, String password);
23 }

```

- la couche WEB qui gère les interactions entre le serveur et les clients.

* Classe ModeleBoraq :

ModeleBoraq une classe qui centralise les informations liées aux réservations et aux entités connexes (voyageurs, trains, stations, horaires). Elle interagit avec d'autres parties de l'application, telles que le contrôleur et les vues, pour afficher et manipuler ces données.

```

1 package com.webapp;
2 import SDIDproject.*;
3 import java.util.ArrayList;
4 import java.util.LinkedList;
5 public class ModeleBoraq {
6     private String nome_porteur;
7     private String searchKeyword;
8     private ArrayList<Reservation> reservations = new ArrayList<>();
9     private ArrayList<Voyageur> voyageurs = new ArrayList<>();
10    private ArrayList<Train> trains = new ArrayList<>();
11    private ArrayList<Station> stations = new ArrayList<>();
12    private LinkedList<Horaire> horaires = new LinkedList<>();
13    private String errorMessage;
14    private Reservation selectedReservation = new Reservation();
15    private String dateChoisie;
16    private String gareDepart;
17    private String gareArrivee;
18    private LinkedList<Horaire> retourHoraires = new LinkedList<>();

```

```

19     private String retourDateChoisie;
20     private double Prixadult ;
21     private double Prixenfant;
22     private int numAdults;
23     private int numChildren;
24     private String email_login;
25     private String password_login;
26     private CarteBancaire carteBancaire;

```

* Classe ControleurBoraq :

ControleurBoraq c'est un controleur responsable de la gestion des interactions de l'utilisateur, de la logique métier et de l'interface utilisateur. Le contrôleur joue un rôle clé dans la séparation des préoccupations, en orchestrant les différentes actions dans une application web. controleurboraq.png

```

1  package com.webapp;
2  import ...
3  @WebServlet(name = "BoraqApp", urlPatterns = {
4      "/accueil", "/horaires", "/voyageursboraq", "/voyageurshoraires",
5      "/registerAction", "/loginAction", "/ClientInfo", "/processPayment"
6  })
7  public class ControleurBoraq extends HttpServlet {
8      private ImplReservation metier;
9      @Override
10     public void init() throws ServletException {
11         metier = new ImplReservation();
12     }
13     @Override
14     protected void doGet(HttpServletRequest request, HttpServletResponse
15         response)
16         throws ServletException, IOException {
17         doPost(request, response); // Handle GET requests the same as POST
18         requests
19     }
20     @Override
21     protected void doPost(HttpServletRequest request, HttpServletResponse
22         response)
23         throws ServletException, IOException {
24         String path = request.getServletPath();
25         ModeleBoraq modele = new ModeleBoraq();
26         try {
27             switch (path) {
28                 case "/accueil":
29                     handleAccueil(request, response, modele);
30                     break;
31                 case "/horaires":
32                     handleHoraires(request, response, modele);
33                     break;
34                 case "/voyageursboraq":
35                     handleVoyageursBoraq(request, response, modele);
36                     break;
37                 case "/voyageurshoraires":
38                     handleVoyageursHoraires(request, response, modele);
39                     break;
40                 case "/registerAction":
41                     handleRegister(request, response);

```

```

39         break;
40     case "/loginAction":
41         handleLogin(request, response);
42         break;
43     case "/ClientInfo":
44         handleClientInfo(request, response);
45         break;
46     case "/processPayment":
47         handleProcessPayment(request, response);
48         break;
49     default:
50         response.sendError(HttpServletResponse.SC_NOT_FOUND, "La page demandée est introuvable.");
51         break;
52     }
53 } catch (SQLException | CompartimentException e) {
54     e.printStackTrace();
55     modele.setErrorMessage("Une erreur est survenue lors de la r cup ration des donn es.");
56     request.setAttribute("modele", modele);
57     request.getRequestDispatcher("accueil.jsp").forward(request, response);
58 }
59 }

```

→ Concernant la partie visible par l'utilisateur, est responsable de l'apparence et de l'interaction. Elle s'appuie sur une architecture composée de deux éléments principaux :

- ***Fichier.JSP*** : Ces fichiers ont été employés pour générer des pages web dynamiques. En combinant du code HTML avec du code Java, ils permettent de créer des interfaces interactives. Les fichiers JSP sont exécutés côté serveur afin de produire un contenu adapté avant d'être transmis au client.

```

1  <html lang="fr">
2  <head>
3      <meta charset="UTF-8">
4      <meta name="viewport" content="width=device-width, initial-scale=1.0">
5      <title>R&eacute;servation de Train</title>
6      <link rel="stylesheet" href="accueilstyle.css">
7  </head>
8  <body>
9  <header>
10     <div class="header-container">
11         <a href="/" class="logo">
12             
16             </a>
17         <nav class="header-nav">
18             <ul class="nav-list">
19                 <li class="nav-item"><a href="https://siana.ma/" class="nav-
20                     link active">ACCUEIL</a></li>

```

```

18         <li class="nav-item"><a href="https://siana.ma/presentation
19         /" class="nav-link">SIANA</a></li>
20         <li class="nav-item"><a href="https://siana.ma/rame-grande-
21         vitesse-maroc/" class="nav-link">ACTIVIT&Eacute;S</a></li>
22         <li class="nav-item"><a href="https://siana.ma/category/
23         evenements/" class="nav-link">ACTUALIT&Eacute;S</a></li>
24         <li class="nav-item"><a href="https://siana.ma/pourquoi-
25         nous-rejoindre/" class="nav-link">CARRI&Egrave;RE</a></li>
26         <li class="nav-item"><a href="https://siana.ma/contact/" cl
27         ass="nav-link">CONTACT</a></li>
28     </ul>
29 </nav>
30 </div>
31 </header>
32 <!-- Suite du code -->

```

- **Fichier.CSS** Ces fichiers ont été utilisés pour définir et personnaliser le style des pages web. Grâce à eux, nous avons pu contrôler l'apparence des éléments HTML, notamment la mise en page, les couleurs, les polices, et la disposition des différents composants.

```

1 :root {
2     --primary-color: #28a745;
3     --secondary-color: #fff;
4     --background-overlay: rgba(0, 0, 0, 0.6);
5 }
6
7 body {
8     margin: 0;
9     font-family: Arial, sans-serif;
10    background-size: cover;
11    background-position: center;
12    background-attachment: fixed;
13    min-height: 100vh;
14 }
15
16 /* Header Styles */
17 .header-container {
18     display: flex;
19     align-items: center;
20     justify-content: space-between;
21     background-color: rgba(255, 255, 255, 0.8); /* Couleur blanche avec 80%
22     d'opacit */
23     padding: 1rem 2rem;
24     box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
25     backdrop-filter: blur(5px); /* Ajoute un effet de flou l'arri re-pl
26     an */
27 }
28
29 .logo-img {
30     height: 50px;
31 }
32
33 .header-nav {
34     flex-grow: 1;

```

```

34     display: flex;
35     justify-content: flex-end;
36 }
37
38 .nav-list {
39     display: flex;
40     gap: 1.5rem;
41     list-style: none;
42     margin: 0;
43     padding: 0;
44 }
45
46 .nav-item {
47     position: relative;
48     height: 100%;
49 }
50
51 .nav-link {
52     font-size: 1rem;
53     color: #333;
54     text-decoration: none;
55     padding: 0.5rem 1rem;
56     transition: color 0.3s, background-color 0.3s;
57     border-radius: 4px;
58 }
59
60 .nav-link.active,
61 .nav-link:hover {
62     color: white;
63     background-color: var(--primary-

```

3 Résultats

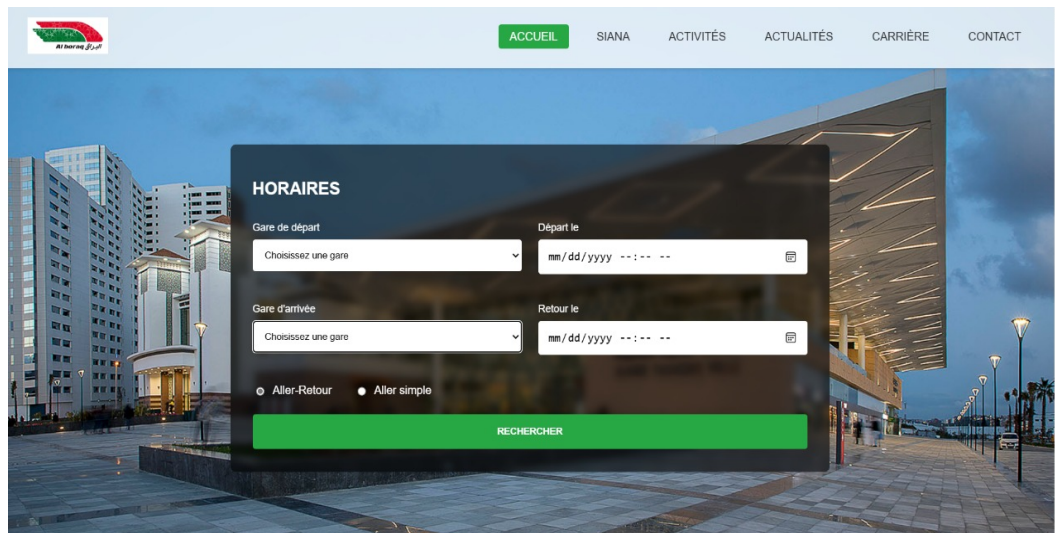
3.1 Tests :

3.1.1 Interface 1 :

Page web dédiée à la recherche d'horaires de train "AL BORAQ".

Une interface utilisateur pour rechercher des trajets de train :

- "Gare de départ" : un menu déroulant permettant de sélectionner une gare de départ.
- "Gare d'arrivée" : un menu déroulant permettant de sélectionner une gare d'arrivée.
- "Départ le" et "Retour le" : des champs de saisie avec un calendrier intégré pour entrer les dates et heures de départ et de retour.
- Options de recherche : Aller-retour ou Aller simple. Un bouton vert marqué Rechercher pour lancer la recherche.



3.1.2 Interface 2 :

Page web dédiée à afficher les horaires des trains -Départ : L'heure de départ des trains. - Arrivée : L'heure d'arrivée prévue . Service à bord : Des icônes représentant des services, ici un symbole de flocon de neige, probablement pour indiquer un train climatisé ou confortable. Gamme : Affiche le logo du service ferroviaire "Al boraq". Bouton d'achat : Une colonne avec un bouton vert marqué "ACHETER EN LIGNE", permettant d'acheter le billet directement.

Départ le : 08/01/2025		Relation : CASA VOYAGEURS - TANGER VILLE		
Départ	Arrivée	Service à bord	Gamme	
15:00:00	17:10:00	❄️		ACHETER EN LIGNE
16:00:00	18:10:00	❄️		ACHETER EN LIGNE
17:00:00	19:10:00	❄️		ACHETER EN LIGNE
18:00:00	20:10:00	❄️		ACHETER EN LIGNE
19:00:00	21:10:00	❄️		ACHETER EN LIGNE
20:00:00	22:10:00	❄️		ACHETER EN LIGNE
21:00:00	23:10:00	❄️		ACHETER EN LIGNE
22:00:00	00:10:00	❄️		ACHETER EN LIGNE

3.1.3 Interface 3 :

Après avoir choisi les horaires et vérifié les disponibilités des trains, cette page est dédiée à l'achat des billets. Elle guide l'utilisateur pour finaliser sa réservation et effectuer le paiement en ligne ,elle contient le détail de mon trajet. -Gares de départ et d'arrivée (par exemple : Casa Voyageurs → Tanger Ville). -Date et heure du trajet sélectionné. -Classe choisie (1re ou 2e classe). -Nombre de passagers (adultes, enfants, etc.).

Acheter mon billet

Pour mieux apprécier vos voyages, l'ONCF met à votre disposition chaque jour et à tout moment de la journée des trains et des autocars vers vos destinations préférées.

Ils desservent toutes les villes du Royaume avec des trajets directs ou avec correspondances. Réservez vos billets sans tarder et pensez à anticiper vos achats pour bénéficier des meilleurs prix.

Bon voyage sur nos lignes !

Mon trajet *

CASA VOYAGEURS TANGER VILLE

Mes dates

mm/dd/yyyy mm/dd/yyyy

Je choisis mon confort

1re classe 2ème classe

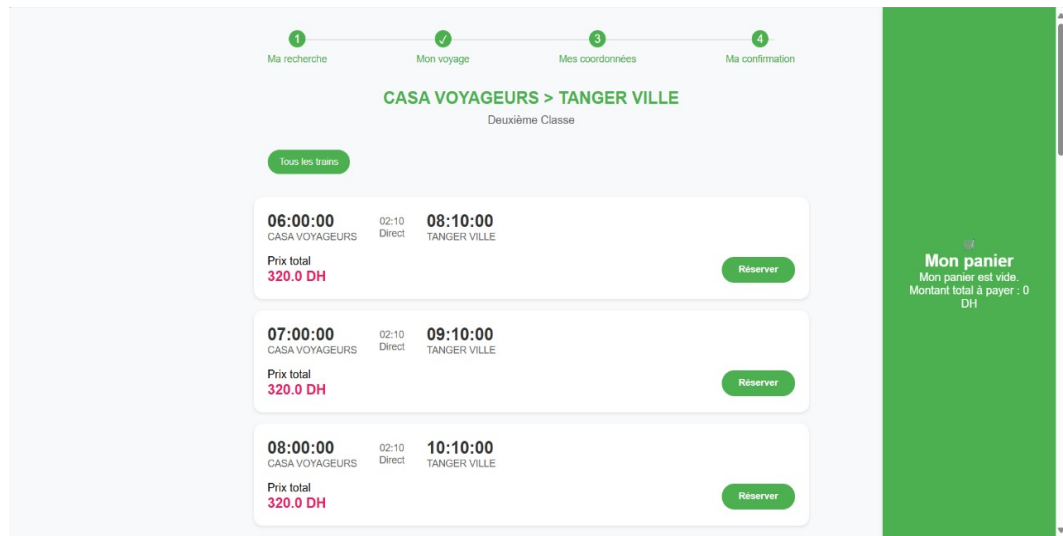
Je choisis le nombre de passagers

Adulte

Rechercher

3.1.4 Interface 4 :

Cette interface permet à l'utilisateur de visualiser les options disponibles pour son trajet (horaires et prix), de comparer et de sélectionner l'option qui lui convient le mieux avant de passer à l'étape suivante de la réservation.



3.1.5 Interface 5 :

Cette page permet l'identification pour accéder à l'espace client. Un utilisateur qui n'a pas de compte peut en créer un afin de procéder au paiement de la réservation.

The screenshot shows a login and registration form titled 'Je souhaite'. It features two tabs: 'M'identifier' (selected) and 'Je crée mon compte'. The form includes input fields for 'Prénom', 'Nom', 'Âge', 'Email', and 'Numéro de téléphone', each with a placeholder text. A green 'Créer un compte' button is at the bottom. The background shows the same progress bar as the previous interface, with step 2 'Mon voyage' checked.

3.1.6 Interface 6 :

Si notre client possède déjà d'un compte, il suffit juste de vérifier les informations de ce dernier et procéder au paiement de la réservation.

1 Ma recherche 2 Mon voyage 3 Mes coordonnées 4 Ma confirmation

Je souhaite

M'identifier Je crée mon compte

Email
Enter your email

Mot de passe
Enter your password

Se connecter

3.1.7 Interface 7 :

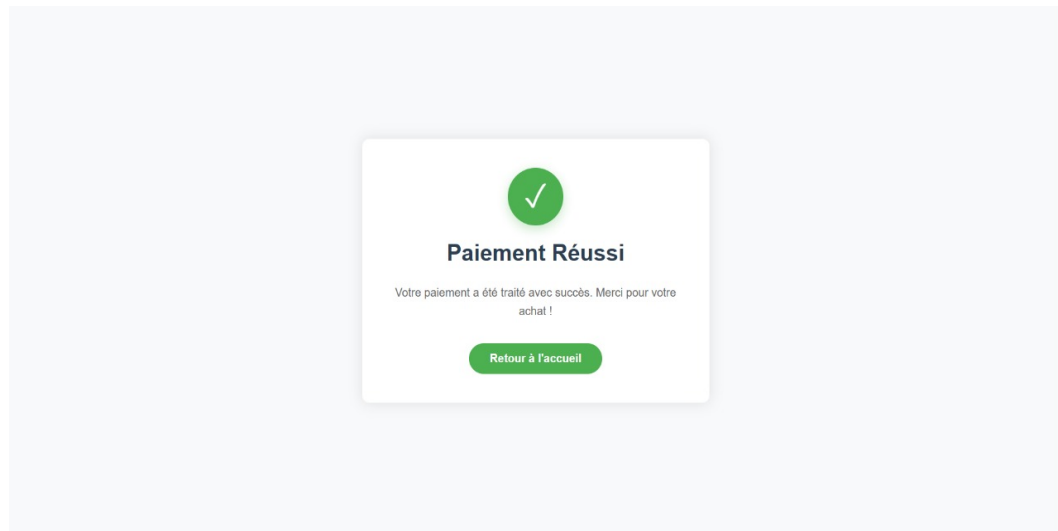
Cette interface représente une page de paiement avec des détails concernant la commande . Elle permet à l'utilisateur d'effectuer un paiement en renseignant les informations de sa carte bancaire. Elle inclut des champs pour :

- Le nom du porteur de la carte
- Le numéro de carte
- La date d'expiration
- Le code de vérification (CVV).

Elle affiche également un résumé des détails de la commande, tels que l'identifiant, le montant, le marchand et les informations du client.

3.1.8 Interface 8 :

Après que l'utilisateur valide son paiement ,un message s'affiche sur l'interface pour informer le client du succès ou de l'échec de son paiement.



3.1.9 Interface 9 :

Après validation du paiement, le ticket sera envoyé à l'adresse mail du client, comme illustré par l'affichage suivant :

```
===== Ticket PDF =====
Ticket #1736165994164
Voyageur: Yasser Bichri
Train: Boraq100
Horaire: CASA VOYAGEURS to RABAT AGDAL
Heure de départ: 06:00:00
Heure d'arrivée: 06:45:00
Compartiment: 3
Place: 1
Prix: 60.0 DH
=====
```

4 Discussion

4.1 Points Forts de notre Projet :

- ✓ **Architecture robuste et évolutive** : Utilisation de Jakarta EE pour une gestion modulaire et adaptable aux besoins futurs.
- ✓ **Interface intuitive** : Navigation simple et fluide, accessible à un large éventail d'utilisateurs.
- ✓ **Gestion optimisée** : Suivi clair des utilisateurs et des réservations en temps réel.
- ✓ **Système modulaire** : Facilité d'ajout de nouvelles fonctionnalités sans perturber l'existant.
- ✓ **Haute fiabilité** : Back-end optimisé pour traiter de multiples requêtes avec sauvegardes automatiques.

4.2 Perspectives

✗ Fonctionnalités supplémentaires :

- Intégration de notifications en temps réel pour alerter les utilisateurs des retards ou promotions.
- Ajout d'options complémentaires, telles que la réservation de repas ou d'hébergement.

✗ Optimisation de l'expérience utilisateur :

- Développement d'une application mobile pour un accès simplifié sur smartphones et tablettes.

→ Modernisation de l'interface pour une navigation encore plus fluide et agréable.

✕ Renforcement de la fiabilité :

→ Mise en place de sauvegardes automatiques pour prévenir toute perte de données.

→ Optimisation du back-end pour une meilleure gestion des requêtes simultanées.

✕ Sécurité renforcée :

→ Sécurisation des informations sensibles grâce à des protocoles avancés.

→ Mise en place de l'authentification multifactorielle pour une protection optimale des comptes.

✕ Internationalisation :

→ Adaptation de l'application pour plusieurs langues et devises afin d'élargir son audience internationale.

4.3 Défis rencontrés

Parmi les défis rencontrés dans notre projet :

- ✓ **Contrainte de temps :** apprendre de nouvelles technologies ou framework tout en respectant les délais peut être stressant.
- ✓ **Conception d'une interface utilisateur :** créer une interface utilisateur agréable peut être difficile sans expérience en HTML et CSS.

5 Conclusion

À travers l'implémentation des fonctionnalités, nous avons essayé d'approfondir notre compréhension d'un système de réservation modernes parmi plusieurs. Ce projet a constitué une passerelle entre la théorie et la pratique, en nous permettant de mettre en œuvre une interface simple et intuitive, tout en garantissant une gestion fluide des fonctionnalités essentielles. Il a permis de relier les concepts étudiés à un projet concret, apportant une réelle valeur ajoutée à l'apprentissage.

Ce projet a ainsi constitué une passerelle entre la théorie et la pratique, en nous permettant de mettre en œuvre une interface simple et intuitive, tout en garantissant une gestion fluide des fonctionnalités essentielles. Il a permis de relier les concepts étudiés à un projet concret, apportant une réelle valeur ajoutée à l'apprentissage.